2026-11-03 회원가입과 인증 참고 가이드

- 날짜: 2026-11-03
- 앱: lgcns-fe-next
- 패키지: next 16.4.3, react 19.3.0, next-auth ^4.24.15, @prisma/client와 prisma ^6.19.3, zod ^4.6.7 (lgcns-fe-next/package.json)
- 기준: 지금 디스크의 파일은 "현재 코드"다. 그 아래에 "가이드"를 따로 적는다. 가이드에만 있는 구조는 소스에 구현된 것이 아니다.
- 값: .env 값, 연결 문자열, NEXTAUTH_SECRET, 비밀번호, 토큰 문자열은 적지 않는다.

이 가이드는 회원가입 단계, 가입 초안, 로그인 진행 상태를 브라우저 쿠키로 이어가지 않는다. document.cookie로 읽거나, 앱이 cookies().set으로 Set-Cookie 하는 방식 모두 가입 진행에는 쓰지 않는다. 권장 구조는 원시 <form action={서버 함수}>와 주소의 search param step뿐이다.

lgcns-fe-next 소스에는 <script> 태그가 없다. Next가 TypeScript 모듈을 묶는다. "use client"와 "use server"는 HTML의 async, defer, type="module"이 아니다.

1. 가이드가 권하는 단계 구조

이 절은 권장 구조다. 아래 hidden input 전달은 현재 소스에 없다.

단계는 URL 하나다. 주소는 /signup?step=1부터 /signup?step=4다. 페이지 서버 컴포넌트는 searchParams의 step만 보고 그 단계의 폼 하나를 그린다. 초안 쿠키를 읽지 않는다. 단계 번호는 쿠키의 step 필드와 맞추지 않는다.

각 단계 폼은 next/form이 아니다. 서버 컴포넌트 안의 원시 <form action={서버 함수}>다. 제출하면 브라우저가 FormData를 만들고, 그 객체가 서버 함수의 인자가 된다. 서버 함수는 formData.get으로만 값을 읽는다. cookies().set을 호출하지 않는다.

이전 단계 값은 쿠키에 쌓지 않는다. 다음 단계 폼이 hidden input으로 다시 실어 보내고, 서버 함수가 그 FormData를 검사한 뒤 redirect로 다음 step을 연다.

값	가이드에서 두는 곳	쿠키에 두지 않는 이유
단계 번호	search param step만	화면이 보여주는 단계는 주소와 같아야 한다. 쿠키의 step과 주소가 어긋나면 새로고침 결과가 주소와 달라진다
userId, email	다음 단계 form의 hidden input. 주소에 옮겨도 되는 식별자면 search param에 실어 다음 페이지가 hidden의 값으로 다시 그린다	가입이 끝나기 전의 입력이다. 로그인 세션이 아니다
password	그 입력이 있는 form의 필드. 이후 단계가 다시 보내야 하면 hidden input이다. 주소의 search param에는 넣지 않는다	쿼리는 히스토리와 서버 로그에 남는다. 쿠키에 넣어도 가입 진행 상태가 요청마다 붙는다
interests	hidden input 또는 search param	선택 입력이고, 다음 제출의 FormData에 있으면 된다
profileImage	마지막 단계 form의 파일 필드. multipart/form-data	파일 바이트를 쿠키나 쿼리에 넣지 않는다
오류 문구	서버 함수가 같은 단계로 redirect 할 때 search param으로 짧은 코드를 넘기거나, 그 응답의 폼 렌더에 직접 그린다	signup-errors 같은 두 번째 쿠키가 필요 없다

호출은 이렇게 이어진다.

1. 브라우저가 /signup?step=1을 연다.
2. 서버 컴포넌트가 step만 보고 1단계 폼을 그린다.

3. 제출된 FormData의 userId, email을 서버 함수가 검사한다.
4. 통과하면 redirect("/signup?step=2") 하고, 2단계 폼은 1단계 값을 hidden input으로 가진다. 그 hidden은 1단계 값을 search param으로 받아 그리거나, 1단계 폼이 이미 갖고 있던 필드를 그대로 둔다.
5. 2단계 FormData에는 password와 hidden으로 넘어온 userId, email이 있다.
6. 3단계, 4단계도 같은 방식이다. 마지막 제출의 FormData에 만들 사용자 필드가 모두 있을 때 userService.create를 호출하고 redirect("/login") 한다.

이 구조에서 가입 진행 상태를 위한 document.cookie와 Set-Cookie는 없다.

2. 현재 코드가 쿠키에 쌓는 곳

현재 코드는 1절을 따르지 않는다. 단계 번호는 주소의 step에도 있지만, 입력값은 actions/draft.ts가 쿠키에 저장한다. 이 쿠키는 잘 만든 가입 구조로 따라 하지 않는다.

쿠키를 쓰는 파일은 두 곳이다.

- lgcns-fe-next/actions/draft.ts. next/headers의 cookies()로 이름을 직접 정한다. signup-draft, signup-errors. 옵션은 httpOnly: true, sameSite: "lax", path: "/", maxAge: 60 * 30이다. document.cookie는 이 파일에 없다. 서버가 Set-Cookie로 넣는 코드다.
- lgcns-fe-next/lib/readAccessToken.ts. cookies()로 요청에 실린 쿠키를 모아 getToken에 넘긴다. 가입 초안이 아니다. 4절의 NextAuth 세션 쿠키를 읽는 경로다.

signup-draft에 들어가는 SignupDraft 필드는 userId, email, identityOk, password, passwordOk, interestsText, interests, interestsConfirmed, step이다. password도 이 JSON에 들어간다. signup-errors의 키는 userId, email, password, interests, profileImageUrl, form이다.

가이드에서는 이 값을 쿠키에 두지 않는다. 대체는 다음 단계 form의 hidden input이거나, 단계 번호와 같이 주소의 search param이다. password와 profileImage는 search param에 올리지 않고 hidden input 또는 파일 필드로 FormData에만 둔다.

현재 쿠키 필드	가이드의 대체
signup-draft.step	search param step만
signup-draft.userId, email, interests, interestsText	다음 단계 form의 hidden input. 필요하면 같은 이름의 search param
signup-draft.password	비밀번호 단계 form의 필드 또는 이후 form의 hidden input. search param 아님
signup-draft.identityOk, passwordOk, interestsConfirmed	서버 함수가 이번 FormData를 검사해 통과할 때만 다음 step으로 redirect. 플래그 쿠키를 만들지 않음
signup-errors의 메시지 필드	같은 단계 주소의 search param 또는 그 응답 폼의 오류 문구

읽는 쪽은 app/(auth)/signup/page.tsx의 readSignupDraft, readSignupErrors다. 쓰는 쪽은 actions/userActions.ts의 writeSignupDraft, writeSignupErrors, clearSignupState다. 페이지는 이 쿠키와 furthestSignupStep으로 주소의 step을 깎은 뒤, 쿼리와 다르면 redirect 한다. 가이드는 그 쿠키 읽기를 단계 조건에 넣지 않는다. 잘못된 step은 주소만 보고 1로 두거나, 이번 FormData에 이전 값이 없으면 그 단계 form을 다시 그린다.

SignupSection이 초안 props로 defaultValue를 채우는 것도 현재 코드다. 가이드의 defaultValue는 쿠키 JSON이 아니라 이번 요청의 search param이나, 방금 제출된 폼을 다시 그릴 때의 값이다.

3. 현재 회원가입 호출

아래는 현재 파일이 하는 일이다. 1절의 가이드와 다르다.

SignupPage는 "use client"가 없는 async function이다. searchParams는 Promise라서 await 한다. 이 await는 함수 안의 일시 중단이지, 스크립트 태그의 async가 아니다. SignupContent가 쿠키 초안을 읽고 SignupSection에 step, draft, errors를 넘긴다. SignupSection은 step이 1이면 SignupIdentityForm, 2이면 SignupPasswordForm, 3이면 SignupInterestsForm, 4이면 SignupProfileForm만 그린다.

폼 태그는 component/molecules/SignupStepActions.tsx에 있다.

```
<form action={submitAction} noValidate encType={encType} className="flex flex-col gap-4">
```

submitAction은 actions/userActions.ts의 서버 함수다. 파일 첫 줄이 "use server"다. next/form import는 소스에 없다. 설치된 node_modules/next/dist/client/app-dir/form.js 첫 줄은 'use client'다. next/form의 Form은 클라이언트 컴포넌트라서, 그 컴포넌트를 쓰면 폼 자체가 클라이언트 경계가 되고 제출 뒤 클라이언트 탐색을 얹는다. 현재 가입 폼은 그 컴포넌트를 쓰지 않는다. 서버 컴포넌트가 서버 함수를 action에 넘긴다. 이 form 선언은 가이드와 같다. 다른 점은 서버 함수가 처리 결과를 쿠키에 쓴다는 점이다.

FormData에서 현재 코드가 빼는 키는 다음과 같다.

키	넣는 곳	읽는 함수
userId	1단계 name="userId"	submitSignupIdentity
email	1단계 name="email"	submitSignupIdentity
password	2단계 name="password"	submitSignupPassword
interests	3단계 name="interests"	submitSignupInterests
profileImage	4단계 파일 입력	submitSignupProfile
profileImageBlocked	4단계 hidden	profileImageFromForm
profileImageUrl	화면 입력에는 없음. 서버가 get만 함	profileImageFromForm
backStep	이전 버튼 name / value	backSignupStep

2단계 이후 폼에는 앞 단계 userId나 email의 hidden input이 없다. 그 값은 signup-draft에 있다. 가이드는 그 hidden을 폼에 넣고, 이 쿠키 키에는 넣지 않는다.

submitSignupIdentity는 형식 오류가 없는 필드만 userService.checkIdentity에 넘긴다. 아이디는 prisma.$runCommandRaw의 find, 이메일은 prisma.user.findUnique다. 통과해도 현재 코드는 사용자를 바로 만들지 않고 쿠키에 identityOk: true를 쓴 뒤 /signup?step=2로 간다. 가이드라면 이 통과 결과를 쿠키 플래그로 남기지 않고, 다음 폼의 hidden에 userId와 email을 실어 같은 주소로 간다.

finishSignup은 쿠키 초안을 다시 읽어 userService.create를 호출한다. 성공하면 clearSignupState()로 쿠키 두 개의 maxAge를 0으로 만들고 redirect("/login") 한다. signIn은 호출하지 않는다. 가이드라면 create의 인자는 마지막 제출의 FormData이고, 지울 가입 쿠키는 없다.

prisma/schema.prisma의 User에는 userId, email, password, interests, profileImageUrl이 있다. 생성된 node_modules/.prisma/client의 UserScalarFieldEnum은 id, email, name, password, createdAt, updatedAt만 있다. userId가 생성 결과에 없어서 아이디 조회는 findUnique가 아니라 $runCommandRaw다. 스키마와 클라이언트를 맞추는 명령 이름은 npx prisma generate다. 이 문서를 쓰면서 실행하지 않았다.

없는 파일: component/organisms/SignupFields.tsx, app/(auth)/signup/actions.ts, app/(auth)/signup/draft.ts. 쿠키 모듈은 actions/draft.ts에 있다.

4. NextAuth

package.json의 "next-auth": "^4.24.15"와 NextAuthOptions, next-auth/providers/credentials import로 보면 메이저는 v4다. Google, GitHub provider는 소스에 없다.

연결은 다섯 파일이다.

1. app/api/auth/[...nextauth]/route.ts가 NextAuth(authOptions)를 GET과 POST로 export한다.
2. lib/authOptions.ts의 session.strategy는 "jwt", maxAge는 60 * 60 * 2초, pages.signIn은 "/login"이다.
3. LoginForm.tsx는 "use client"다. signIn("credentials", { email: payload.userId, password: payload.password, redirect: false })를 호출한다. 폼 name은 userId이고, Credentials 필드는 email이다.
4. authorize가 userService.login에 넘긴 결과에서 id, email, name, accessToken, expiresIn을 반환한다. jwt 콜백이 token.accessToken, token.expiresIn, token.id에 복사한다. session 콜백이 session.accessToken, session.expiresIn, session.user의 id, email, name에 복사한다. 타입은 types/next-auth.d.ts다. profileImageUrl은 타입에만 있고 콜백 반환에는 없다.
5. AuthShell은 getServerSession(authOptions)로 session.accessToken을 AuthProvider에 넘긴다. readAccessToken을 import하지만 호출하지 않는다. /login 페이지는 readAccessToken → getToken으로 액세스 토큰 문자열이 있으면 redirect("/") 한다.

v4 JWT 전략의 세션 쿠키 이름은 설치된 next-auth/core/lib/cookie.js에 있다. 기본 이름은 next-auth.session-token이고, secure 쿠키면 앞에 __Secure-가 붙는다. 이 쿠키는 라우트 핸들러가 로그인 세션을 저장하려고 만드는 것이다. 앱 코드가 가입 단계의 userId, email, password, interests, profile을 이 쿠키에 넣거나, 그 옆에 signup-draft를 추가하는 패턴으로 쓰지 않는다. NextAuth 자체를 빼라는 뜻이 아니다. 로그인 세션 저장은 이 프레임워크 쿠키에 두고, 회원가입 진행 상태는 1절처럼 form과 step에만 둔다.

document.cookie로 세션 토큰을 읽거나 쓰는 코드는 lgcns-fe-next 가입·로그인 파일에 없다. readAccessToken의 cookies()는 프레임워크 세션 쿠키를 getToken에 넘기는 현재 코드다. 가입 필드를 같은 방식으로 Set-Cookie 하라는 뜻이 아니다.

app/api/auth/login/route.ts의 JSON POST는 LoginForm이 호출하지 않는다. 작업 트리의 lgcns-fe-html/js/login.js와 lgcns-fe/src/lib/auth.ts가 이 경로를 fetch하고 sessionStorage에 액세스 토큰을 넣는다. sessionStorage도 가입 단계 저장소로 가이드하지 않는다. 가입 단계는 form과 search param이다.

5. 커스텀 Credentials

providers에는 CredentialsProvider만 있다. OAuth처럼 외부 프로필로 사용자를 만들지 않는다. credentials는 email(type: "text")과 password다. authorize는 둘 다 있을 때만 userService.login을 호출하고, 실패하면 null이다. login은 prisma.user.findUnique({ where: { email } })와 timingSafeEqual이다. 폼에 적은 문자열이 이메일일 때 이 조회와 맞는다.

성공 시 accessToken은 randomBytes(32).toString("hex")이고 expiresIn은 60 * 60 * 2다. 그 값은 적지 않는다. authorize, userService.login, AuthShell, AuthProvider의 console.log도 값과 해시를 이 문서에 옮기지 않는다. login의 select에는 password가 들어 있다.

이 프로바이더가 커스텀인 지점은 userService.login의 반환을 JWT와 세션의 accessToken, expiresIn, id에 싣는 부분이다. 가입 네 단계를 이 세션 쿠키에 나누어 저장하는 코드는 없다.

6. CSR과 SSR

두 페이지 파일 모두 서버 컴포넌트다. 가입 가이드의 SSR은 search param과 form action이다. 브라우저 쿠키는 권장 저장소가 아니다.

로그인 app/(auth)/login/page.tsx는 서버에서 getToken을 보고, 문자열이 있으면 redirect("/") 한다. 폼은 LoginForm.tsx의 "use client"다. useState와 preventDefault 뒤 signIn이라 제출만 클라이언트 요청이다. 성공하면 router.replace("/")와 router.refresh()가 있다. 마운트마다 router.replace 하는 useEffect는 이 파일에 없다. 로그인 입력은 React state다. 로그인 세션 저장은 NextAuth의 next-auth.session-token이고, 가입 초안 쿠키와 합치지 않는다. 가이드는 이 세션 쿠키 옆에 로그인 상태를 또 document.cookie로 두지 않는다.

회원가입 페이지도 서버 컴포넌트다. 현재 코드는 렌더 전에 signup-draft를 읽는다. 가이드의 같은 페이지는 searchParams.step만 읽고, 제출은 서버 함수가 FormData만 읽은 뒤 redirect 한다. 클라이언트에 남는 것은 FormField의 지우기, SignupPhotoInput의 미리보기, SignupSubmitButton의 useFormStatus, SignupProgress의 useSearchParams다. SignupProgress는 쿠키를 읽지 않는다. 주소의 step만 본다. 이 진행 표시는 가이드와 맞다.

비교	로그인 현재 코드	회원가입 현재 코드	회원가입 가이드
페이지 선언	서버 컴포넌트	서버 컴포넌트	서버 컴포넌트
폼	"use client"와 signIn	원시 form action. 처리 뒤 쿠키를 씀	원시 form action. 쿠키를 쓰지 않음
렌더 시점	요청 때 서버. 입력 변화는 브라우저	요청 때 서버. redirect로 다음 문서	요청 때 서버. step이 바뀐 redirect로 다음 문서
상태 위치	입력은 React state. 세션은 NextAuth 세션 쿠키	입력은 signup-draft. 단계는 ?step=과 쿠키 step	단계는 ?step=. 입력은 그 단계 form의 FormData와 hidden input
첫 HTML	빈 로그인 폼. 세션이 있으면 폼 없이 redirect	쿠키에서 온 defaultValue와 오류	search param과 hidden으로 그린 그 단계 폼
새로고침	세션 쿠키가 있으면 서버가 /로 보냄	가입 쿠키가 남아 단계가 주소와 따로 움직일 수 있음	주소의 step과 그 요청의 쿼리·폼만 보면 된다

서버 컴포넌트의 이점은 폼 마크업이 첫 HTML에 있다는 점이다. 가이드에서 그 이점을 살리는 저장은 쿠키가 아니라 step과 form action이다. 허용되지 않은 단계는 서버가 주소만 보고 되돌린다. 이전 값은 다음 요청의 쿼리와 hidden에 있으므로 signup-draft 없이도 서버가 같은 폼을 다시 그릴 수 있다.

클라이언트 폼의 이점은 로그인에 있다. 글자마다 loginSchema로 버튼을 잠그고, 실패 문구를 페이지 재요청 없이 바꾼다. 가입의 지우기와 사진 미리보기만 클라이언트로 두는 현재 분할은 가이드에서도 그대로 둘 수 있다. 그 클라이언트 상태가 가입 원본은 아니다.

Next.js는 서버 컴포넌트를 기본으로 두고 "use client"를 클라이언트 경계로 둔다. 서버 함수는 "use server" export다. 이 저장소에서 그 세 파일은 app/(auth)/signup/page.tsx, LoginForm.tsx, actions/userActions.ts다. 가이드는 세 번째 파일이 FormData와 redirect만 하게 두고, draft.ts의 cookies().set은 따르지 않는다.

7. 커밋된 파일과 미커밋 변경

체크아웃은 feature/signup-stepper-formaction-step3, 팁은 15532ec다. actions/draft.ts는 git status의 수정 목록에 없다. signup-draft를 쓰는 코드는 팁에 이미 있다. 작업 트리만 다른 Next 파일은 actions/userActions.ts, app/(auth)/signup/page.tsx, SignupProgress.tsx, SignupIdentityForm.tsx, SignupFormSection.tsx, SignupSection.tsx다.

팁의 페이지는 SignUpFormSection과 next/link의 다음·이전 Link를 그린다. 페이지 안 "use server" 함수는 console.log(formData)만 한다. 작업 트리 페이지는 SignupSection을 그리고 쿠키 초안과 ?step=을 맞춘다. 팁의 SignupFormSection은 @base-ui/react/form으로 1단계 폼만 그린다. 작업 트리는 SignupSection을 반환하지만 페

이지가 이 파일을 import하지 않는다. 팁의 SignupIdentityForm은 서버 함수를 연결하지 않는다. 작업 트리는 submitSignupIdentity를 action에 넘긴다. 팁의 userActions.ts는 redirect("/signup")이고, 작업 트리는 /signup?step= 에 숫자를 붙인다. FormData 키와 쿠키 기록은 팁에도 있다.

따라서 쿠키 초안은 커밋된 현재 동작이고, 그 쿠키를 페이지 렌더에 연결한 것이 미커밋 차이 중 하나다. 가이드는 어느 쪽도 가입 저장소로 따르지 않는다. 가이드가 따르는 현재 코드는 원시 form action, FormData 키 이름, SignupProgress의 useSearchParams다.

lgcns-fe-html과 lgcns-fe의 로그인 fetch는 미커밋이다. Next signIn과 다른 경로다.

8. 브랜치가 가리키는 커밋

구현 설명은 현재 파일과 1절 가이드다. 브랜치는 git branch -vv의 커밋만 적는다.

브랜치	커밋	원격
feature/signup-stepper-formaction-step3 (현재)	15532ec	origin과 같음. 7장의 미커밋은 이 커밋 밖
feature/signup-stepper-formaction	56d5f2c	원격 추적 있음
feature/signup-stepper-formaction-step2	56d5f2c 같은 커밋	원격 추적 없음
feature/signup-step1	ad13628	원격 추적 있음
main	01eb35b	origin/main보다 1커밋 앞
origin/main	f9618fc	원격 기본 브랜치
feat/feed-searchparams	dc16abb	2026-10-30
feat/keopupi-rule	15f5dd2	2026-10-30

9. 환경 키와 스크립트 로딩

lib/prisma.ts와 scripts/with-database-url.mjs는 APP_ENV가 "develop"이면 키 이름 DATABASE_URL_DEVELOP, 그 기본이면 DATABASE_URL_LOCAL을 읽어 process.env.DATABASE_URL에 넣는다. 스키마는 DATABASE_URL만 본다. lgcns-fe-next/.env에 있는 키 이름은 APP_ENV, NEXTAUTH_SECRET이다. DATABASE_URL, DATABASE_URL_LOCAL, DATABASE_URL_DEVELOP, NEXTAUTH_URL 키 이름은 이 파일에 없다. 값은 적지 않는다.

package.json의 db:push는 node --env-file=.env ./scripts/with-database-url.mjs db push다. .mjs는 ES 모듈이다. import { spawn } from "node:child_process" 뒤에 Prisma CLI를 spawn한다. 이 명령은 실행하지 않았다.

lgcns-fe/.env는 추적되지 않는다. 키 이름은 VITE_APP_ENV, VITE_API_BASE_LOCAL, VITE_API_BASE_DEVELOP다.

파일	선언	로딩
lgcns-fe-html/login.html	</body> 직전 <script src="js/config.js"> 다음 <script src="js/login.js">	고전 스크립트. async 없음, defer 없음, type="module" 없음. 동기, 파서 차단, 문서 순서
lgcns-fe-html/index.html, about.html, contact.html	config.js, auth-status.js, main.js	같은 고전 동기 태그. contact.html에도 async는 없다
lgcns-fe/index.html	<script type="module" src="/src/main.tsx">	ES 모듈. 모듈 스크립트는 defer와 같이 파싱 뒤에 순서대로 실행
lgcns-fe-next의 페이지와 userActions.ts	소스에 <script> 없음	"use client", "use server", export async function

login.html의 두 스크립트는 body 끝의 동기 스크립트라 config.js가 window.apiBaseUrl을 만든 뒤에 login.js가 읽는다. 리스너의 async는 태그의 async 속성이 아니다. Next 페이지의 async function도 브라우저 defer가 아니다. SignupPage의 await searchParams는 서버 함수 안의 일시 중단이다.

10. 공식 문서

- MDN script 요소: https://developer.mozilla.org/ko/docs/Web/HTML/Element/script
- HTML 명세, script 요소: https://html.spec.whatwg.org/multipage/scripting.html#the-script-element
- HTML 명세, defer: https://html.spec.whatwg.org/multipage/scripting.html#attr-script-defer
- ECMA-262, 스크립트와 모듈: https://tc39.es/ecma262/multipage/ecmascript-language-scripts-and-modules.html
- MDN FormData: https://developer.mozilla.org/ko/docs/Web/API/FormData
- MDN 버튼 formaction: https://developer.mozilla.org/ko/docs/Web/HTML/Element/button#formaction
- HTML 명세, 폼 제출: https://html.spec.whatwg.org/multipage/form-control-infrastructure.html#form-submission-algorithm
- MDN URLSearchParams: https://developer.mozilla.org/ko/docs/Web/API/URLSearchParams
- React "use client": https://react.dev/reference/rsc/use-client
- React "use server": https://react.dev/reference/rsc/use-server
- React 서버 함수: https://react.dev/reference/rsc/server-functions
- Next.js 서버와 클라이언트 컴포넌트: https://nextjs.org/docs/app/getting-started/server-and-client-components
- Next.js 서버 함수로 데이터 갱신: https://nextjs.org/docs/app/getting-started/updating-data
- Next.js 페이지 searchParams: https://nextjs.org/docs/app/api-reference/file-conventions/page
- Next.js useSearchParams: https://nextjs.org/docs/app/api-reference/functions/use-search-params
- Next.js next/form: https://nextjs.org/docs/app/api-reference/components/form
- Next.js redirect: https://nextjs.org/docs/app/api-reference/functions/redirect
- NextAuth v4 Credentials: https://next-auth.js.org/providers/credentials
- NextAuth v4 signIn: https://next-auth.js.org/getting-started/client#signin
- NextAuth v4 getServerSession: https://next-auth.js.org/configuration/nextjs#getserversession
- NextAuth v4 콜백: https://next-auth.js.org/configuration/callbacks
- NextAuth v4 getToken: https://next-auth.js.org/tutorials/securing-pages-and-api-routes
- Prisma findUnique: https://www.prisma.io/docs/orm/reference/prisma-client-reference#findunique
- Prisma MongoDB $runCommandRaw: https://www.prisma.io/docs/orm/prisma-client/using-raw-sql/raw-queries